

✓ **Projet Epidémiologie Cabanes / Dechenais**

✓ **Chargement des données :**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.utils import resample
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from scipy import stats
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import KNNImputer
```

```
%matplotlib inline
```

```
df_h1 = pd.read_csv('liver.csv')
df_h2 = pd.read_json('liver.json')
```

```
df_h1
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	ALT	AST	Total_Proteins	AI
0	41 to 60 yo	Male	7.0	2.0	174.0	NaN	14	7800.0 mg/dL	
1	41 to 60 yo	Male	6.0	2.0	245.0	22.0	24	7100.0 mg/dL	
2	21 to 40 yo	Male	11.0	5.0	191.0	37.0	41	7700.0 mg/dL	
3	21 to 40 yo	Male	24.0	10.0	340.0	25.0	21	8300.0 mg/dL	
4	21 to 40 yo	Male	6.0	2.0	202.0	NaN	41	8000.0 mg/dL	
...	...	...	...	...	...	...	...	...	
316	<20 yo	Male	14.0	5.0	269.0	58.0	45	6700.0 mg/dL	
317	61 to 80 yo	Male	7.0	1.0	196.0	20.0	35	5800.0 mg/dL	
318	61 to 80 yo	Male	28.0	13.0	250.0	NaN	29	2700.0 mg/dL	
319	41 to 60 yo	Male	9.0	3.0	901.0	23.0	17	6200.0 mg/dL	
320	21 to 40 yo	Male	12.0	3.0	498.0	28.0	25	7000.0 mg/dL	

321 rows × 11 columns

Next steps:

Generate code with df_h1

☒ View recommended plots

New interactive sheet

df_h2

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	As
0	61 to 80 yo	Female	0.7	0.1	187.0		NaN
1	61 to 80 yo	Male	10.9	5.5	699.0		64.0
2	61 to 80 yo	Male	7.3	4.1	490.0		NaN
3	41 to 60 yo	Male	1.0	0.4	182.0		14.0
4	61 to 80 yo	Male	3.9	2.0	195.0		27.0
...	...	...	...	...	...		...
286	21 to 40 yo	Female	0.8	0.2	192.0		15.0
287	41 to 60 yo	Male	1.3	0.6	155.0		15.0
288	41 to 60 yo	Female	1.0	0.5	239.0		16.0
289	41 to 60 yo	Male	4.5	2.3	315.0		120.0
290	41 to 60 yo	Female	1.0	0.3	250.0		48.0

291 rows × 11 columns

Next steps:

Generate code with df_h2

 View recommended plots

New interactive sheet

▼ Analyse des jeux de données :

```
print(df_h1.info())
df_h2.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex:321 entries, 0 to 320
```

```
Data columns (total 11 columns):
#      Column                                Non-Null Count  Dtype
---  -
0     Age                                    321 non-null   object
1     Gender                                321 non-null   object
2     Total_Bilirubin                       317 non-null   float64
3     Direct_Bilirubin                      320 non-null   float64
4     Alkaline_Phosphotase                   317 non-null   float64
5     ALT                                    254 non-null   float64
6     AST                                    321 non-null   int64
7     Total_Proteins                         321 non-null   object
8     Albumin                               316 non-null   float64
9     Albumin_and_Globulin_Ratio            262 non-null   float64
10    Result                                321 non-null   int64
```

```
dtypes: float64(6), int64(2), object(3)
```

```
memory usage: 27.7+ KB
```

```
None
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 291 entries, 0 to 290
```

```
Data columns (total 11 columns):
```

```
#      Column                                Non-Null Count  Dtype
---  -
0     Age                                    291 non-null   object
1     Gender                                291 non-null   object
2     Total_Bilirubin                       290 non-null   float64
3     Direct_Bilirubin                      287 non-null   float64
4     Alkaline_Phosphotase                   290 non-null   float64
5     Alamine_Aminotransferase               236 non-null   float64
6     Aspartate_Aminotransferase             291 non-null   int64
7     Total_Proteins                         289 non-null   float64
8     Albumin                               290 non-null   float64
9     Albumin_and_Globulin_Ratio            227 non-null   float64
10    Outcome                                291 non-null   int64
```

```
dtypes: float64(7), int64(2), object(2)
```

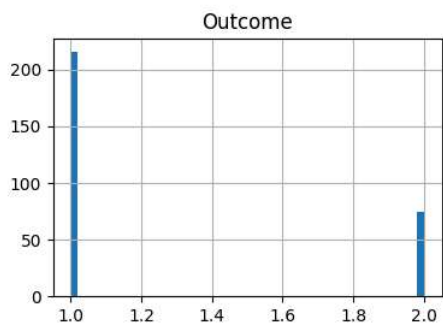
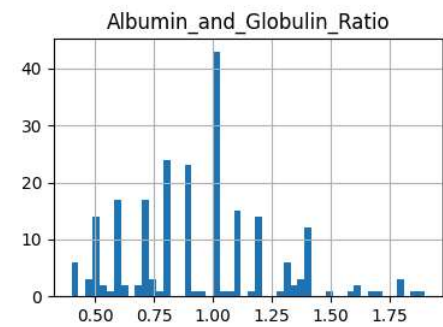
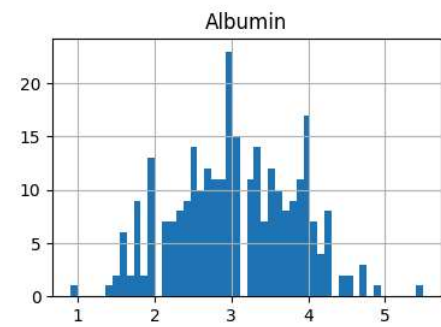
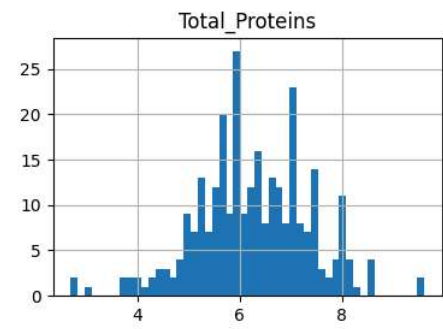
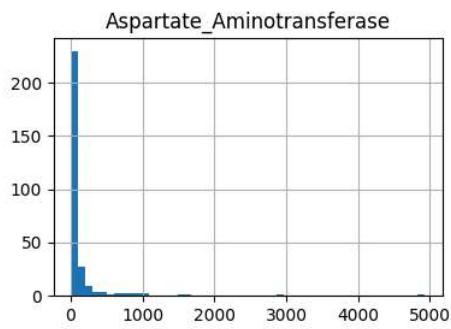
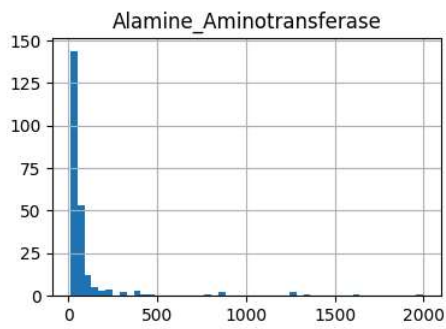
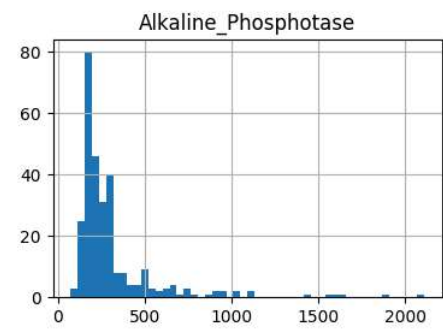
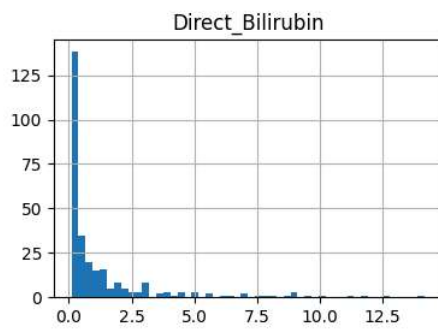
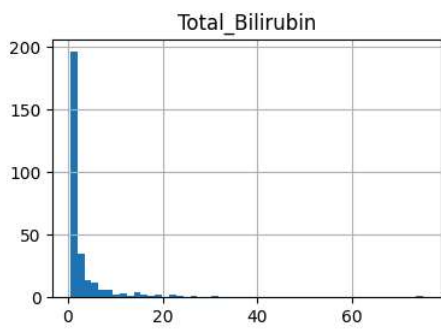
```
memory usage: 25.1+ KB
```

Remarque :

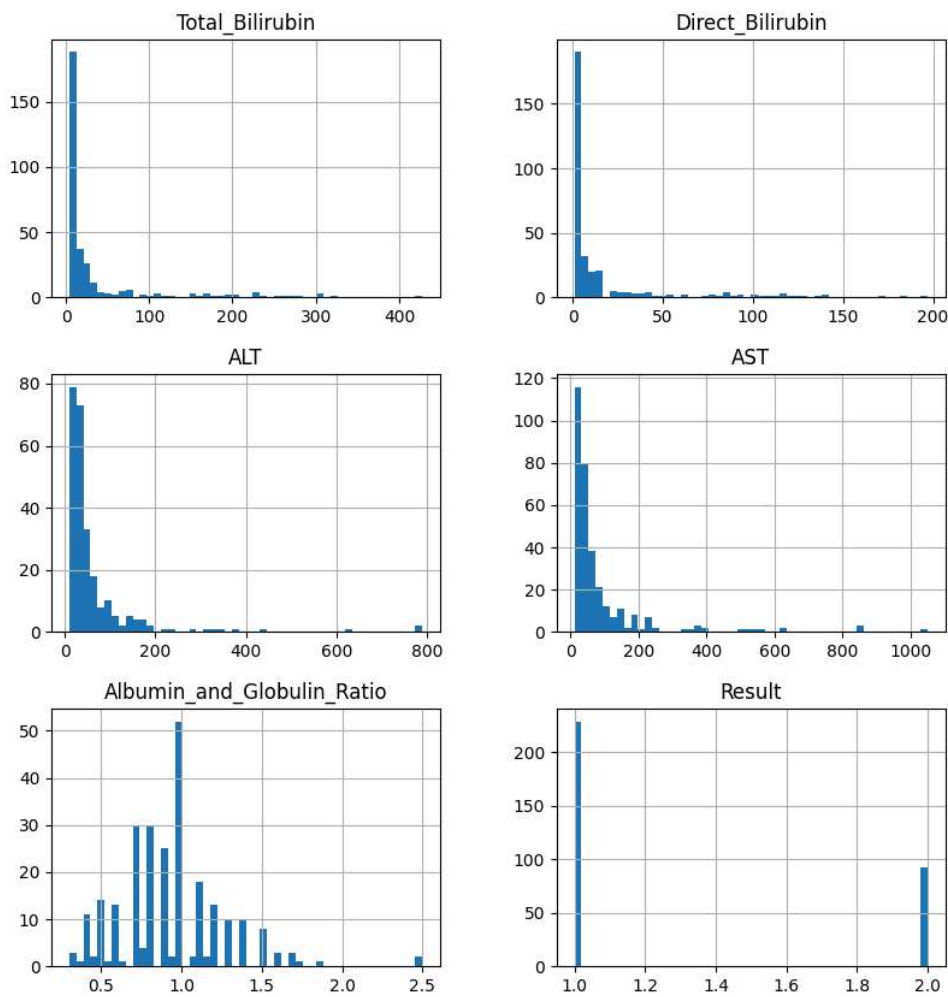
- Total_Proteins est de type objet dans le df_h1 et de type float64 dans le df_h2
- ALT, ASP et Result de df_h1 sont appelés Alamine_Aminotransferase, Aspartate_Aminotransferase et Outcome dans df_h2

✓ Visualisation graphique :

```
df_h2.hist(bins=50, figsize=(15, 10))
plt.show()
```



```
df_h1.hist(bins=50, figsize=(15, 10))  
plt.show()
```



✓ Harmonisation de l'unité de la colonne Total_Proteins :

```
def check_protein_format(protein_str, unite):
    return isinstance(protein_str, str) and protein_str.endswith(unite)

def extract_protein_value(protein_str):
    if check_protein_format(protein_str, 'mg/dL'):
        protein_float = float(str.split(protein_str, ' ')[0])
        return protein_float
    if check_protein_format(protein_str, 'g/dL'):
        protein_float = float(str.split(protein_str, ' ')[0])
        return protein_float * 1000
    if check_protein_format(protein_str, 'nan mg/dL'):
        return np.nan

df_h1['Total_Proteins'] = df_h1['Total_Proteins'].apply(extract_protein_value)
df_h1.head(3)
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphatase	ALT	AST	Total_Proteins	Albu
0	41 to 60 yo	Male	7.0	2.0	174.0	NaN	14	7800.0	
1	41 to 60 yo	Male	6.0	2.0	245.0	22.0	24	7100.0	
2	21 to 40 yo	Male	11.0	5.0	191.0	37.0	41	7700.0	

Next steps:

[Generate code with df_h1](#)

[View recommended plots](#)

[New interactive sheet](#)

Harmonisation de l'unité de Total_Bilirubin, Direct_Bilirubin :

```
def harmonize_Total_Bilirubin(df):
    df['Total_Bilirubin'] = df['Total_Bilirubin']*10
    return df
```

```
df_h2 = harmonize_Total_Bilirubin(df_h2)
df_h2.head(3)
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphatase	Alamine_Aminotransferase	Aspa
0	61 to 80 yo	Female	7.0	0.1	187.0		NaN
1	61 to 80 yo	Male	109.0	5.5	699.0		64.0
2	61 to 80 yo	Male	73.0	4.1	490.0		NaN

Next steps:

[Generate code with df_h2](#)

[View recommended plots](#)

[New interactive sheet](#)

```
def harmonize_Direct_Bilirubin(df):
    df['Direct_Bilirubin'] = df['Direct_Bilirubin']*10
    return df
```

```
df_h2 = harmonize_Direct_Bilirubin(df_h2)
df_h2.head(3)
```



	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspa
0	61 to 80 yo	Female	7.0	1.0	187.0		NaN
1	61 to 80 yo	Male	109.0	55.0	699.0		64.0
2	61 to 80 yo	Male	73.0	41.0	490.0		NaN

Next steps:

[Generate code with df_h2](#)

 [View recommended plots](#)

[New interactive sheet](#)

Renommer les colonnes du jeu de données df_h1 :

```
df_h1.rename(columns={'Age': 'Age'
, 'Gender': 'Gender'
, 'Total_Bilirubin': 'Total_Bilirubin'
, 'Direct_Bilirubin': 'Direct_Bilirubin'
, 'Alkaline_Phosphotase': 'Alkaline_Phosphotase'
, 'ALT': 'Alamine_Aminotransferase'
, 'AST': 'Aspartate_Aminotransferase'
, 'Total_Protiens': 'Total_Protiens'
, 'Albumin': 'Albumin'
, 'Albumin_and_Globulin_Ratio': 'Albumin_and_Globulin_Ratio'
, 'Result': 'Outcome'
}, inplace=True)
```

```
df_h1.head(3)
```



	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspa
0	41 to 60 yo	Male	7.0	2.0	174.0		NaN
1	41 to 60 yo	Male	6.0	2.0	245.0		22.0
2	21 to 40 yo	Male	11.0	5.0	191.0		37.0

Next steps:

[Generate code with df_h1](#)

 [View recommended plots](#)

[New interactive sheet](#)

Concaténation des jeux de données :


```
df_combined = pd.concat([df_h1, df_h2])
df_combined
```



	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	As
0	41 to 60 yo	Male	7.0	2.0	174.0		NaN
1	41 to 60 yo	Male	6.0	2.0	245.0		22.0
2	21 to 40 yo	Male	11.0	5.0	191.0		37.0
3	21 to 40 yo	Male	24.0	10.0	340.0		25.0
4	21 to 40 yo	Male	6.0	2.0	202.0		NaN
...	...	...	...	...	...		...
286	21 to 40 yo	Female	8.0	2.0	192.0		15.0
287	41 to 60 yo	Male	13.0	6.0	155.0		15.0
288	41 to 60 yo	Female	10.0	5.0	239.0		16.0
289	41 to 60 yo	Male	45.0	23.0	315.0		120.0
290	41 to 60 yo	Female	10.0	3.0	250.0		48.0
612 rows × 11 columns							

Next steps:

[Generate code with df_combined](#)

☒ View recommended plots

[New interactive sheet](#)

Retirer les doublons :

```
df_combined.drop_duplicates(inplace=True)
df_combined
```



	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	As
0	41 to 60 yo	Male	7.0	2.0	174.0		NaN
1	41 to 60 yo	Male	6.0	2.0	245.0		22.0
2	21 to 40 yo	Male	11.0	5.0	191.0		37.0
3	21 to 40 yo	Male	24.0	10.0	340.0		25.0
4	21 to 40 yo	Male	6.0	2.0	202.0		NaN
...	...	...	...	...	...		...
286	21 to 40 yo	Female	8.0	2.0	192.0		15.0
287	41 to 60 yo	Male	13.0	6.0	155.0		15.0
288	41 to 60 yo	Female	10.0	5.0	239.0		16.0
289	41 to 60 yo	Male	45.0	23.0	315.0		120.0
290	41 to 60 yo	Female	10.0	3.0	250.0		48.0

606 rows × 11 columns

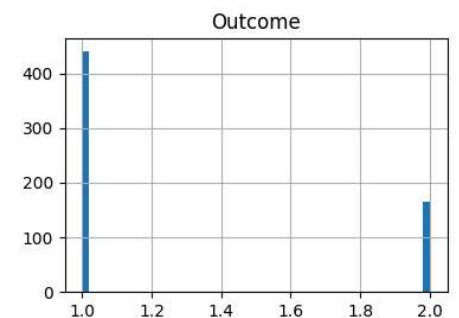
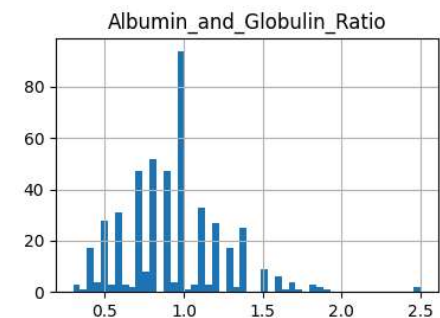
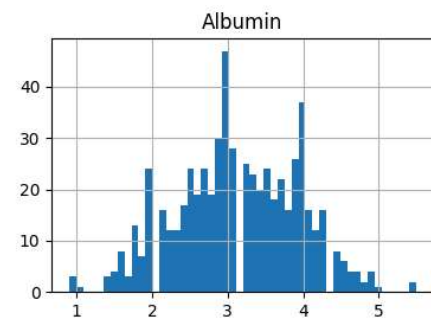
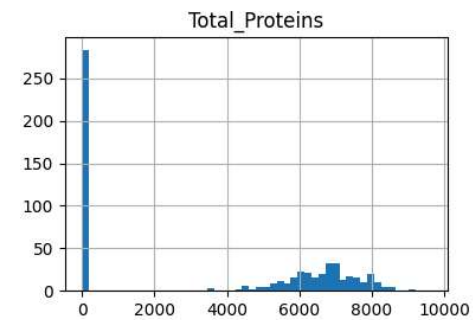
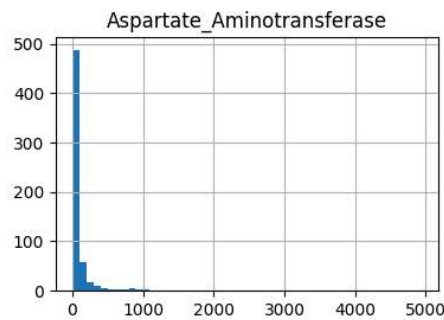
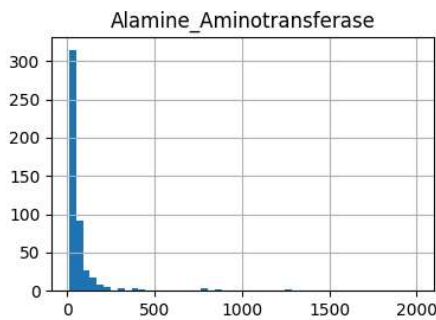
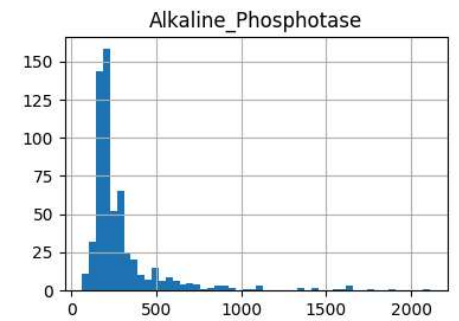
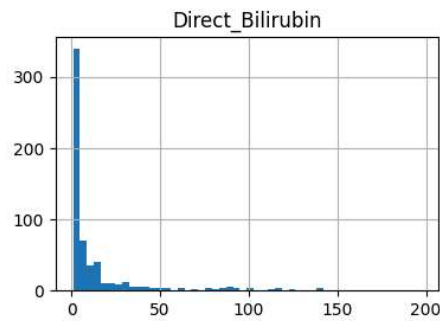
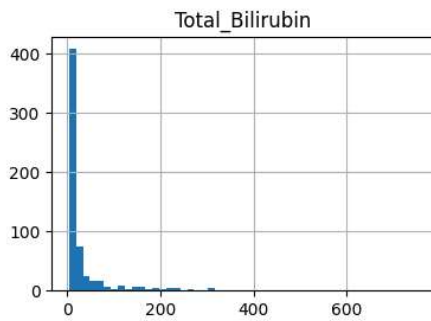


Next steps:

- Generate code with df_combined
- View recommended plots
- New interactive sheet

Visualiser la distribution des données :

```
hist_df_combined = df_combined.hist(bins=50, figsize=(15, 10))
```



Il y a des valeurs abérrantes dans certaines colonnes (Exemple : Total_Proteins avec beaucoup de 0).

✓ Gestion des Variables Catégorielles (OneHotEncoding, puis LabelEncoding) :

j'ai utilisé ce site pour mettre 1 et 0 au lieu de True et False

<https://stackoverflow.com/questions/77643432/why-is-pd-get-dummies-returning-boolean-values-instead-of-the-binaries-of-0-1>

```
df_encoded = pd.get_dummies(data = df_combined, columns=['Gender'], dtype=int)
df_encoded.head(3)
```



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Am
0	41 to 60 yo	7.0	2.0	174.0		NaN
1	41 to 60 yo	6.0	2.0	245.0		22.0
2	21 to 40 yo	11.0	5.0	191.0		37.0

Next steps:

[Generate code with df_encoded](#)

[View recommended plots](#)

[New interactive sheet](#)

```
#LabelEncoder les tris par ordre alphabétique et non pas dans l'ordre que l'on veut.
# On a donc créée une fonction pour le faire manuellement.
d = {'<20 yo': 0, '21 to 40 yo': 1, '41 to 60 yo': 2, '61 to 80 yo': 3, '>80 yo': 4}
def LabelEncoder2(val):
    return d[val]
```

```
df_encoded_2 = df_encoded.copy()
df_encoded_2['Age'] = df_encoded_2['Age'].apply(LabelEncoder2)
```

```
df_encoded_2.head(3)
```

```
# 0 pour <20 yo, 1 pour 21-40 yo, 2 pour 41-60 yo, 3 pour 61-80 yo, 4 pour >80 yo
```

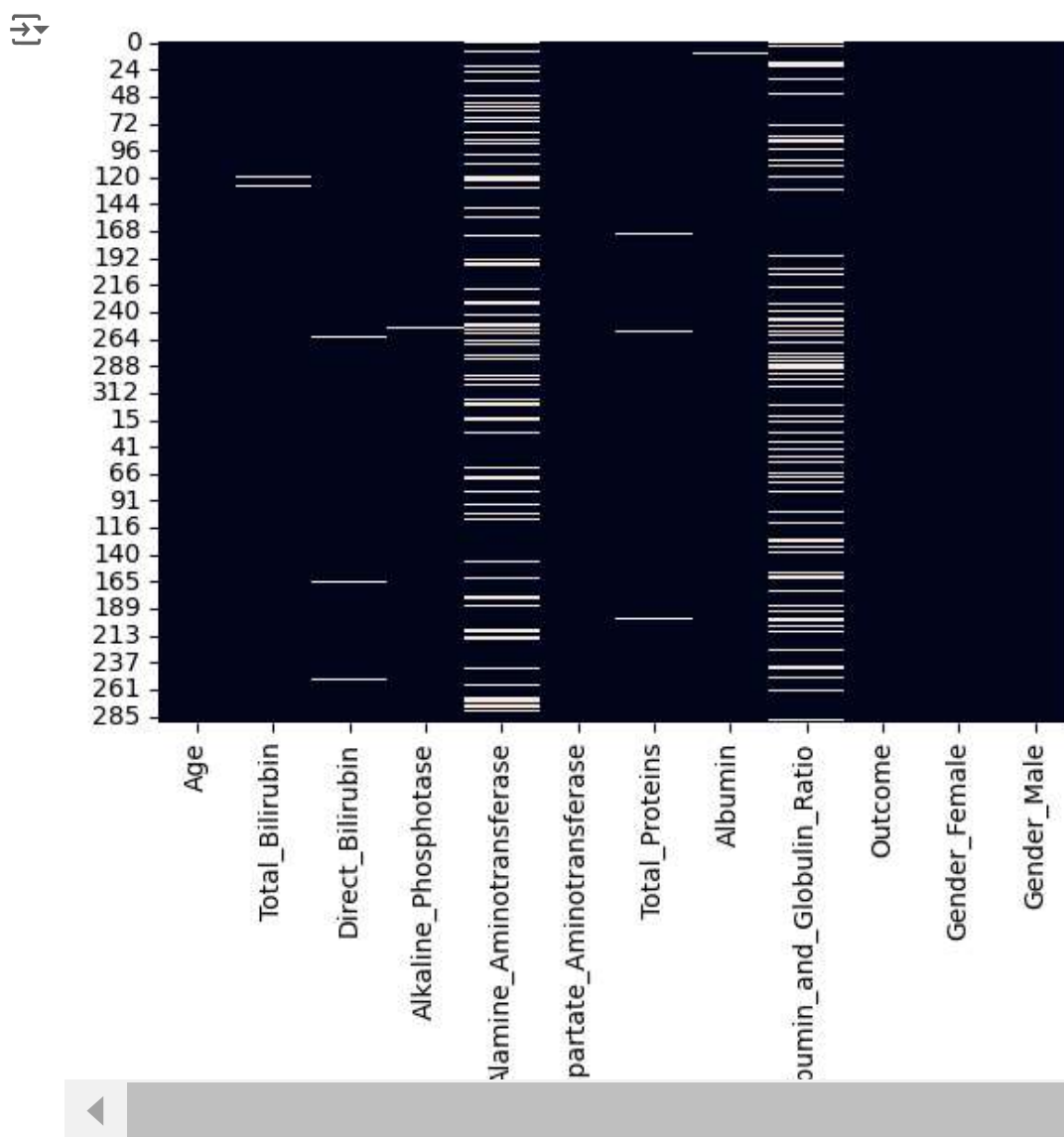


	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Am
0	2	7.0	2.0	174.0		NaN
1	2	6.0	2.0	245.0		22.0
2	1	11.0	5.0	191.0		37.0

✓ Complétion des valeurs manquantes :

```
# Afficher un aperçu des valeurs manquantes (méthode isna())
import seaborn
```

```
na_columns = df_encoded_2.columns[df_encoded_2.isna().any()].tolist()
na_columns2 = seaborn.heatmap(df_encoded_2.isna(), cbar=False)
```



✎ Imputation des valeurs par méthode K-Nearest Neighbour (KNN) :

```
def data_imputation(df, cols = []):
    int_columns = df.select_dtypes(include='int').columns.tolist()
    imputer = KNNImputer(n_neighbors=5)
    df_imputed = df.copy()

    df_imputed[cols] = imputer.fit_transform(df_imputed[cols])

    df_imputed[int_columns] = df_imputed[int_columns].round(0).astype(int)

    return df_imputed

df_imputed = data_imputation(df_encoded_2, na_columns)

df_imputed
```



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_
0	2	7.0	2.0	174.0	58.6	
1	2	6.0	2.0	245.0	22.0	
2	1	11.0	5.0	191.0	37.0	
3	1	24.0	10.0	340.0	25.0	
4	1	6.0	2.0	202.0	47.6	
...	...	...	...	...	...	...
286	1	8.0	2.0	192.0	15.0	
287	2	13.0	6.0	155.0	15.0	
288	2	10.0	5.0	239.0	16.0	
289	2	45.0	23.0	315.0	120.0	
290	2	10.0	3.0	250.0	48.0	

606 rows × 12 columns

Next steps:

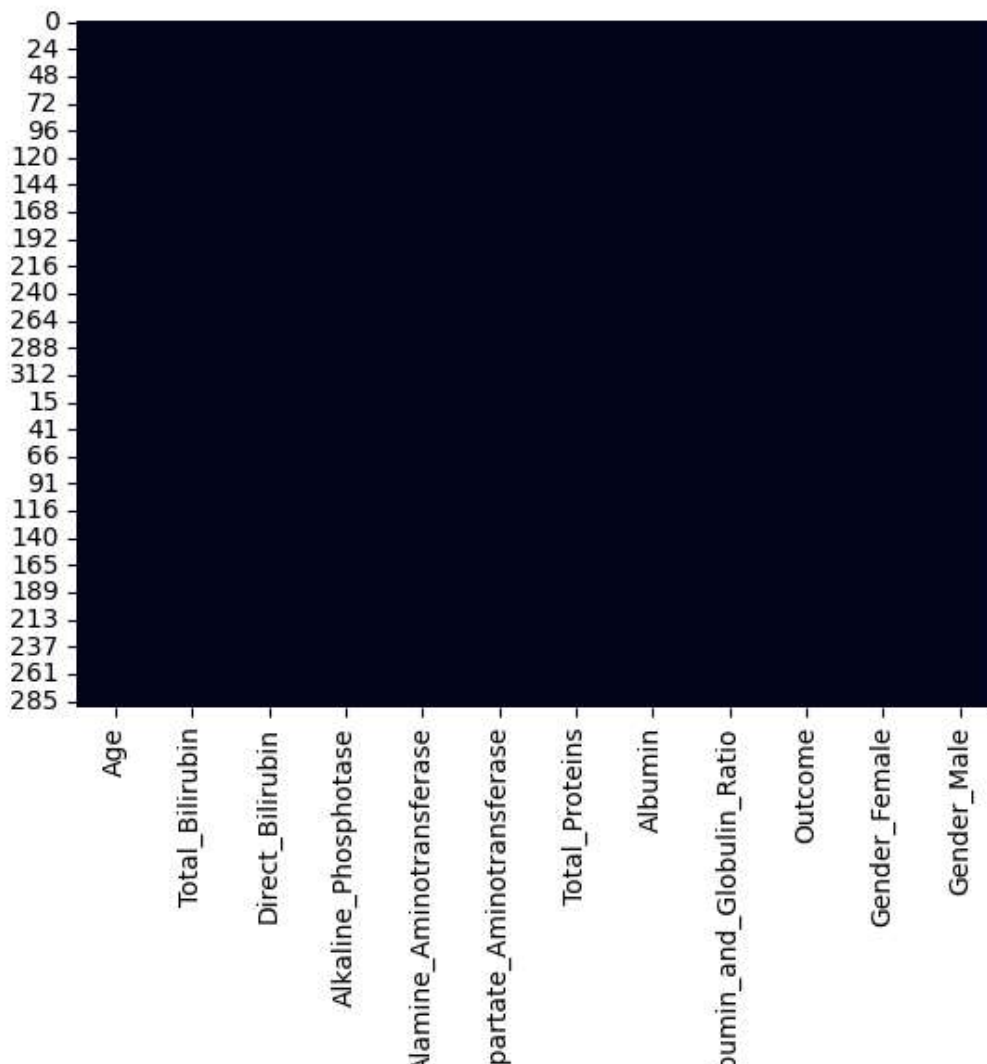
[Generate code with df_imputed](#)

[View recommended plots](#)

[New interactive sheet](#)

```
na_columns = df_imputed.columns[df_imputed.isna().any()].tolist()
```

```
na_columns2 = seaborn.heatmap(df_imputed.isna(), cbar=False)
```



✓ Qualité des données après complétion des données manquantes :

```
def compare_stats(df1, df2):
    stats_diff = pd.DataFrame(index=df1.columns, columns=['Mean_Diff', 'Std_Diff'])

    stats_diff['Mean_Diff'] = ((df2.mean() - df1.mean()) / df1.mean()).abs()
    stats_diff['Std_Diff'] = ((df2.std() - df1.std()) / df1.std()).abs()

    return stats_diff

df_stats_comp_after_completion = compare_stats(df_encoded_2, df_imputed)
```

df_stats_comp_after_completion



	Mean_Diff	Std_Diff	
Age	0.000000	0.000000	
Total_Bilirubin	0.002792	0.003870	
Direct_Bilirubin	0.002453	0.003529	
Alkaline_Phosphotase	0.001439	0.003899	
Alamine_Aminotransferase	0.032667	0.069563	
Aspartate_Aminotransferase	0.000000	0.000000	
Total_Proteins	0.002864	0.002988	
Albumin	0.000175	0.002517	
Albumin_and_Globulin_Ratio	0.005237	0.083362	
Outcome	0.000000	0.000000	
Gender_Female	0.000000	0.000000	

Next steps:

[Generate code with df_stats_comp_after_completion](#)

 [View recommended plots](#)

[New interactive sheet](#)

✓ Biais d'outlier et détection des valeurs aberrantes :

```
def detect_outliers(df, num_std=2, threshold=1):
    df_outliers = df.copy()
    means = df_outliers.mean()
    stds = df_outliers.std()
    df_outliers['Outlier_Count'] = ((df_outliers - means).abs() > num_std * stds).sum(axis=1)

    return df_outliers

def remove_outliers(df, threshold):
    df_with_outliers = detect_outliers(df)
    df_without_outliers = df_with_outliers[df_with_outliers['Outlier_Count'] <= threshold].drop(columns='Ou
    return df_without_outliers

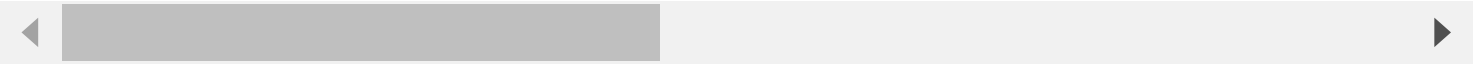
df_outliers = remove_outliers(df_imputed,2)
```

df_outliers



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_
0	2	7.0	2.0	174.0	58.6	
1	2	6.0	2.0	245.0	22.0	
2	1	11.0	5.0	191.0	37.0	
3	1	24.0	10.0	340.0	25.0	
4	1	6.0	2.0	202.0	47.6	
...	...	...	...	...	...	...
286	1	8.0	2.0	192.0	15.0	
287	2	13.0	6.0	155.0	15.0	
288	2	10.0	5.0	239.0	16.0	
289	2	45.0	23.0	315.0	120.0	
290	2	10.0	3.0	250.0	48.0	

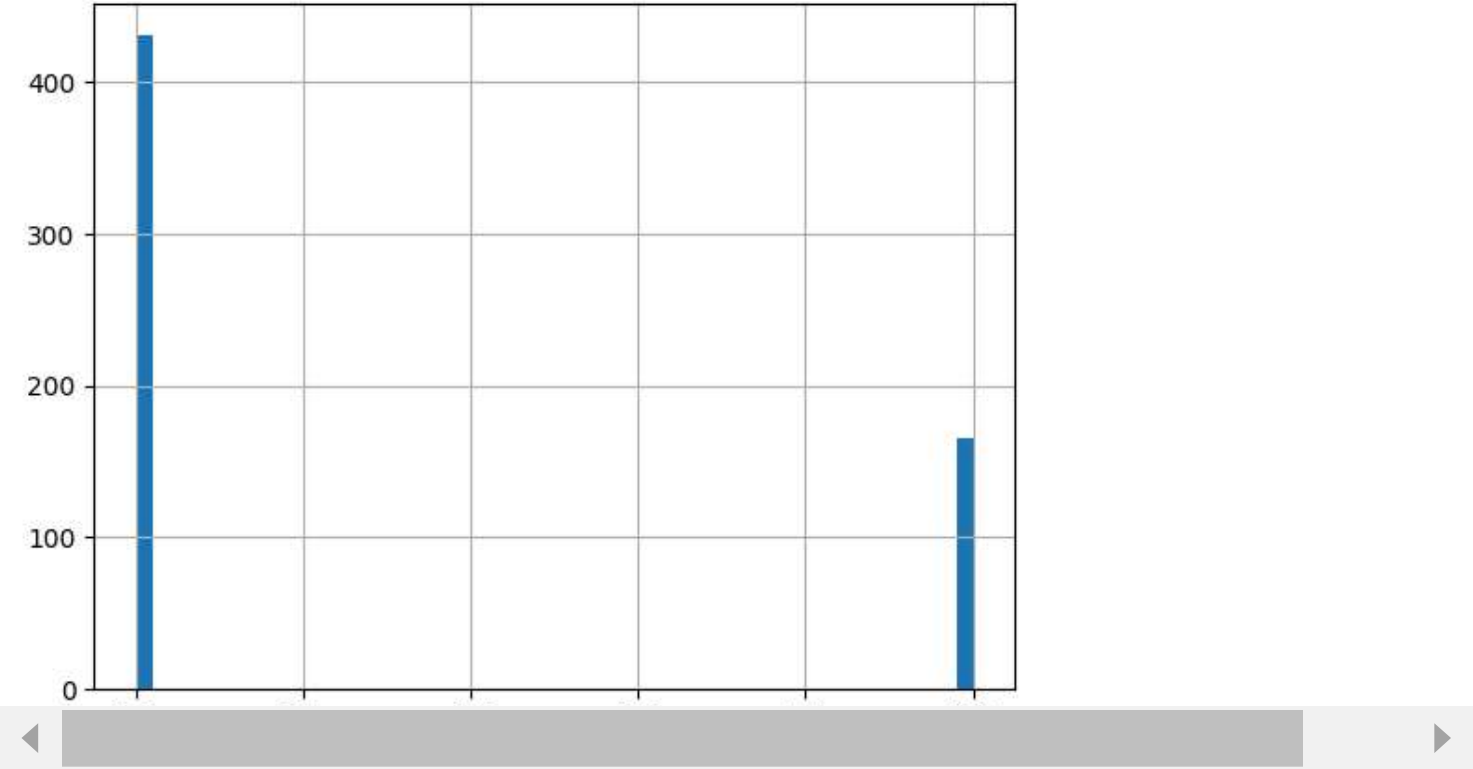
595 rows × 12 columns



Next steps: [Generate code with df_outliers](#) [View recommended plots](#) [New interactive sheet](#)

▼ Gestion du Déséquilibre de Classe :

```
hist_df_outliers = df_outliers['Outcome'].hist(bins=50)
```



df_outliers



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_
0	2	7.0	2.0	174.0	58.6	
1	2	6.0	2.0	245.0	22.0	
2	1	11.0	5.0	191.0	37.0	
3	1	24.0	10.0	340.0	25.0	
4	1	6.0	2.0	202.0	47.6	
...	...	...	...	...	...	...
286	1	8.0	2.0	192.0	15.0	
287	2	13.0	6.0	155.0	15.0	
288	2	10.0	5.0	239.0	16.0	
289	2	45.0	23.0	315.0	120.0	
290	2	10.0	3.0	250.0	48.0	

595 rows × 12 columns

Next steps:

[Generate code with df_outliers](#)



[View recommended plots](#)

[New interactive sheet](#)

```
def get_majority_class_df(df,column):
    df_majority = df[df[column] == 1]
    return df_majority

def get_minority_class_df(df,column):
    df_minority = df[df[column] == 2]
    return df_minority

def oversampling(df):
    df_minority = get_minority_class_df(df,'Outcome')
    df_majority = get_majority_class_df(df,'Outcome')
    df_minority_oversampled = resample(df_minority,
                                       replace=True,
                                       n_samples=len(df_majority),
                                       random_state=42)
    df_oversampled = pd.concat([df_majority, df_minority_oversampled])
    return df_oversampled

def smote(df):
    X = df.drop('Outcome', axis=1)
    y = df['Outcome']
    smote = SMOTE(random_state=42)
    X_smote, y_smote = smote.fit_resample(X, y)
    df_smote = pd.concat([pd.DataFrame(X_smote, columns=X.columns), pd.DataFrame(y_smote, columns=['Outcome'])])
    return df_smote

df_over = oversampling(df_outliers)
df_smote = smote(df_outliers)

df_smote
```



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_
0	2	7.000000	2.000000	174.000000	58.600000	
1	2	6.000000	2.000000	245.000000	22.000000	
2	1	11.000000	5.000000	191.000000	37.000000	
3	1	24.000000	10.000000	340.000000	25.000000	
4	1	6.000000	2.000000	202.000000	47.600000	
...	...	...	...	...	...	...
855	2	10.784778	2.946195	214.139113	20.892389	
856	2	14.000000	6.218704	237.910901	24.844625	
857	2	7.340394	2.113465	178.822991	33.212709	
858	2	7.930929	2.000000	178.756724	30.618582	
859	1	7.103007	2.077255	243.386277	21.695298	

860 rows × 12 columns

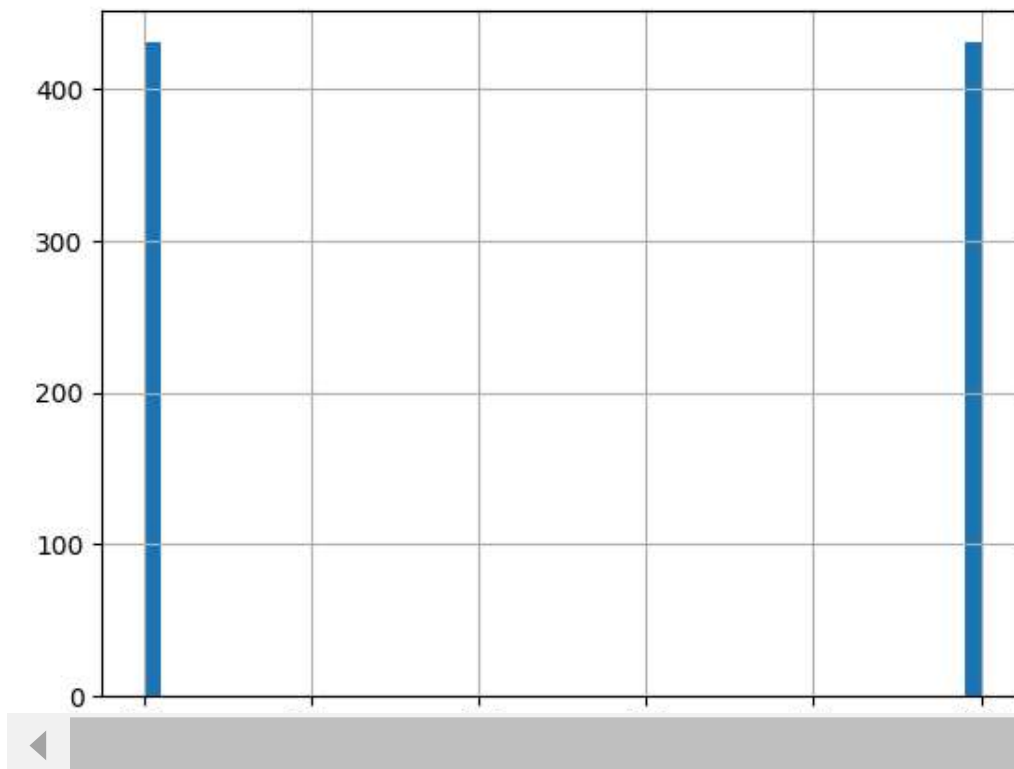
Next steps:

[Generate code with df_smote](#)

[View recommended plots](#)

[New interactive sheet](#)

```
hist_df_smote = df_smote['Outcome'].hist(bins=50)
```



Qualité des données après complétion des données manquantes et gestion du déséquilibre de classe (SMOTE) :

```
def compare_stats(df1, df2):  
    stats_diff = pd.DataFrame(index=df1.columns, columns=['Mean_Diff', 'Std_Diff'])  
  
    stats_diff['Mean_Diff'] = ((df2.mean() - df1.mean()) / df1.mean()).abs()  
    stats_diff['Std_Diff'] = ((df2.std() - df1.std()) / df1.std()).abs()
```

```
return stats_diff
```

```
df_stats_comp_after_completion = compare_stats(df_imputed, df_smote)
```

on remarque de gros écarts entre les deux dataframes... (ce qui n'est pas trop bon signe pour la qualité
df_stats_comp_after_completion



	Mean_Diff	Std_Diff	
Age	0.089143	0.004154	
Total_Bilirubin	0.287873	0.219729	
Direct_Bilirubin	0.320185	0.232038	
Alkaline_Phosphotase	0.117357	0.192244	
Alamine_Aminotransferase	0.260349	0.283274	
Aspartate_Aminotransferase	0.307869	0.414307	
Total_Proteins	0.018940	0.009962	
Albumin	0.029195	0.039583	
Albumin_and_Globulin_Ratio	0.033817	0.083624	
Outcome	0.178988	0.122989	
Gender_Female	0.212161	0.083743	

Next steps:

[Generate code with](#) df_stats_comp_after_completion

 [View recommended plots](#)

[New interactive sheet](#)

```
print(df_imputed['Gender_Female'].value_counts())
print(df_imputed['Gender_Male'].value_counts())

print(df_smote['Gender_Female'].value_counts())
print(df_smote['Gender_Male'].value_counts())

# remarque: la méthode SMOTE a généré des personnes qui ont 0 et 0 ou 1 et 1 dans les colonnes Gender_Femal
```



```
Gender_Female
0    462
1    144
Name: count, dtype: int64
Gender_Male
1    462
0    144
Name: count, dtype: int64
Gender_Female
0    699
1    161
Name: count, dtype: int64
Gender_Male
1    601
0    259
Name: count, dtype: int64
```



Qualité des données après complétion des données manquantes et gestion du déséquilibre de classe (Oversampling) :

```
def compare_stats(df1, df2):
    stats_diff = pd.DataFrame(index=df1.columns, columns=['Mean_Diff', 'Std_Diff'])

    stats_diff['Mean_Diff'] = ((df2.mean() - df1.mean()) / df1.mean()).abs()
    stats_diff['Std_Diff'] = ((df2.std() - df1.std()) / df1.std()).abs()

    return stats_diff

df_stats_comp_after_completion = compare_stats(df_imputed, df_over)

# on remarque également de gros écarts entre les deux dataframes... (ce qui n'est pas trop bon signe pour l
# on va rester sur la méthode smote car c'est celle que l'on avait prise pour l'atelier2.
df_stats_comp_after_completion
```



	Mean_Diff	Std_Diff	
Age	0.032470	0.041898	
Total_Bilirubin	0.268127	0.216647	
Direct_Bilirubin	0.298647	0.227823	
Alkaline_Phosphotase	0.100594	0.176174	
Alamine_Aminotransferase	0.220171	0.271698	
Aspartate_Aminotransferase	0.270047	0.412101	
Total_Proteins	0.007589	0.015893	
Albumin	0.023905	0.029330	
Albumin_and_Globulin_Ratio	0.033059	0.004976	
Outcome	0.178988	0.122989	
Gender_Female	0.115698	0.036793	

Next steps:

Generate code with

df_stats_comp_after_completion

☐ View recommended plots

[New interactive sheet](#)

```
print(df_imputed['Gender_Female'].value_counts())
print(df_imputed['Gender_Male'].value_counts())

print(df_over['Gender_Female'].value_counts())
print(df_over['Gender_Male'].value_counts())

print(df_smote['Gender_Female'].value_counts())
print(df_smote['Gender_Male'].value_counts())
```



```
Gender_Female
0    462
1    144
Name: count, dtype: int64
Gender_Male
1    462
0    144
Name: count, dtype: int64
Gender_Female
0    632
```

```

1      228
Name: count, dtype: int64
Gender_Male
1      632
0      228
Name: count, dtype: int64
Gender_Female
0      699
1      161
Name: count, dtype: int64
Gender_Male
1      601
0      259
Name: count, dtype: int64

```

✓ Biais de représentation des classes dans les groupes de données :

```
print(df_smote.groupby('Gender_Female')['Outcome'].value_counts(normalize=True))
```

```
print(df_smote.groupby('Gender_Male')['Outcome'].value_counts(normalize=True))
```

#il va y avoir un biais dans les données, mais malheureusement, on ne peut pas faire grand chose pour l'évi

```

⇒ Gender_Female Outcome
0          2      0.516452
          1      0.483548
1          1      0.571429
          2      0.428571
Name: proportion, dtype: float64
Gender_Male Outcome
0          2      0.644788
          1      0.355212
1          1      0.562396
          2      0.437604
Name: proportion, dtype: float64

```

✓ Normalisation des Données :

pour faire cela, on a décidé de faire passer l'outcome de 1 ou 2 à 0 ou 1 soit :

```
df_smote['Outcome'] = df_smote['Outcome'] - 1
```

```
from sklearn.preprocessing import MinMaxScaler
```

```

def min_max_scaling(df):
    minMaxScaler = MinMaxScaler()
    colonneOutcome = df['Outcome']
    autresColonnes = df.drop(columns=['Outcome'])

    autresColonnesNorm = minMaxScaler.fit_transform(autresColonnes)
    df_normalized = pd.DataFrame(autresColonnesNorm, columns=autresColonnes.columns)
    df_normalized['Outcome'] = colonneOutcome
    return df_normalized

```

```

df_normalized = min_max_scaling(df_smote)
df_normalized

```



	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate
0	0.50	0.004021	0.005102	0.054226	0.024422	
1	0.50	0.002681	0.005102	0.088911	0.006030	
2	0.25	0.009383	0.020408	0.062531	0.013568	
3	0.25	0.026810	0.045918	0.135320	0.007538	
4	0.25	0.002681	0.005102	0.067904	0.018894	
...	...	...	...	...	...	...
855	0.50	0.009095	0.009930	0.073834	0.005474	
856	0.50	0.013405	0.026626	0.085447	0.007460	
857	0.50	0.004478	0.005681	0.056582	0.011665	
858	0.50	0.005269	0.005102	0.056549	0.010361	
859	0.25	0.004160	0.005496	0.088122	0.005877	

860 rows × 12 columns



Next steps:

[Generate code with df_normalized](#)



[View recommended plots](#)

[New interactive sheet](#)

```
from sklearn.preprocessing import StandardScaler
```

```
def standardisation(df):
```

```
    scaler = StandardScaler()
```

```
    outcome = df['Outcome']
```

```
    features = df.drop(columns=['Outcome'])
```

```
    features_standardized = scaler.fit_transform(features)
```

```
    df_standardized = pd.DataFrame(features_standardized, columns=features.columns)
```

```
    df_standardized['Outcome'] = outcome
```

```
    return df_standardized
```

```
df_standardized = standardisation(df_smote)
```

```
df_standardized
```

	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspar
0	0.524928	-0.344281	-0.377406	-0.427591	0.005564	
1	0.524928	-0.364532	-0.377406	-0.077265	-0.298222	
2	-0.644600	-0.263277	-0.241101	-0.343710	-0.173720	
3	-0.644600	-0.000011	-0.013926	0.391482	-0.273322	
4	-0.644600	-0.364532	-0.377406	-0.289434	-0.085738	
...	...	...	...	...	...	...
855	0.524928	-0.267635	-0.334416	-0.229538	-0.307416	
856	0.524928	-0.202523	-0.185729	-0.112244	-0.274611	
857	0.524928	-0.337388	-0.372251	-0.403794	-0.205155	
858	0.524928	-0.325429	-0.377406	-0.404120	-0.226686	
859	-0.644600	-0.342195	-0.373896	-0.085227	-0.300751	

860 rows × 12 columns

Next steps:

Generate code with df_standardized

View recommended plots

New interactive sheet

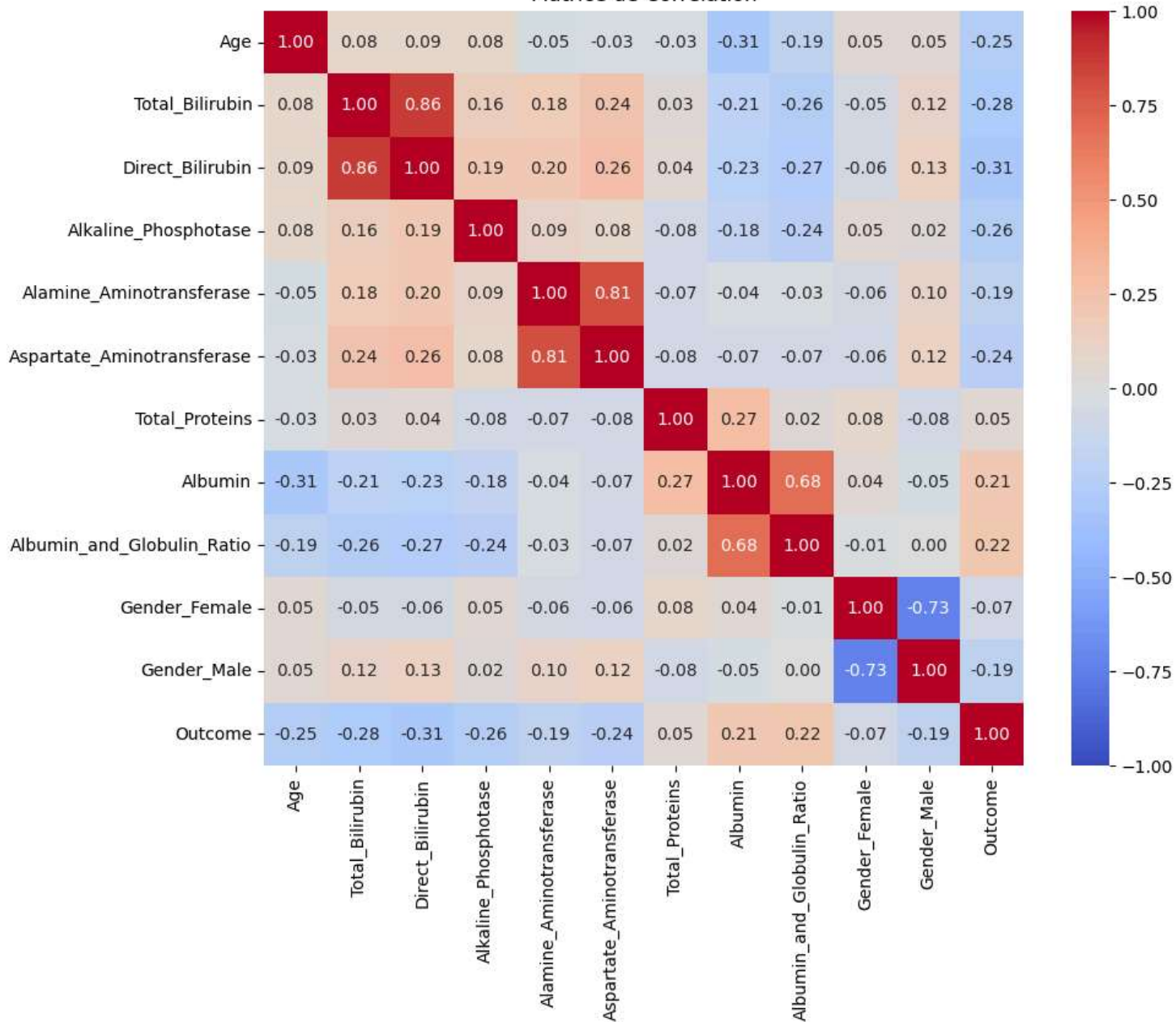
J AI UN PROBLEME AVEC SMOTE QUI ME CREER DES PERSONNES SANS GENRE

✓ Visualisation des Corrélations :

```
plt.figure(figsize=(10, 8))
sns.heatmap(df_normalized.corr(), annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Matrice de Corrélation')
plt.show()
```



Matrice de Corrélation



✓ Sélection de Features sur Base de la Matrice de Corrélation :

j'avais fait ce prompt pour l'atelier2 : Écris une fonction en Python pour sélectionner les colonnes les plus corrélées avec la variable cible dans un DataFrame standardisé. Trie les corrélations par ordre décroissant, filtre les colonnes ayant une corrélation absolue supérieure à 0.2, et retourne un nouveau DataFrame contenant uniquement ces colonnes.

```
def select_features(df):  
    # 1. Calcule la corrélation absolue avec 'Outcome' et trie les résultats  
    corr_target = df_standardized.corr()['Outcome'].abs().sort_values(ascending=False)  
  
    # 2. Sélectionne les features ayant une corrélation > 0.2  
    selected_features = corr_target[corr_target > 0.2].index.tolist()  
  
    # 3. Affiche les features sélectionnées  
    print(selected_features)
```



```
# 4. Retourne un nouveau DataFrame avec uniquement les features sélectionnées
return df[selected_features]
```

```
df_standardized.to_csv('CABANES_DECHENAIIS-PROJET.csv')
```

 ['Outcome', 'Direct_Bilirubin', 'Total_Bilirubin', 'Alkaline_Phosphotase', 'Age', 'Aspartate_Aminotransfere

	Outcome	Direct_Bilirubin	Total_Bilirubin	Alkaline_Phosphotase	Age	Aspartate_Aminotransfere
0	1	-0.377406	-0.344281	-0.427591	0.524928	-0.37
1	0	-0.377406	-0.364532	-0.077265	0.524928	-0.31
2	1	-0.241101	-0.263277	-0.343710	-0.644600	-0.21
3	0	-0.013926	-0.000011	0.391482	-0.644600	-0.33
...	...	...	...	...	0	-0.21
...	...	...	...	...	...	...
855	1	-0.334416	-0.267635	-0.229538	0.524928	-0.36
856	1	-0.185729	-0.202523	-0.112244	0.524928	-0.33
857	1	-0.372251	-0.337388	-0.403794	0.524928	-0.29
858	1	-0.377406	-0.325429	-0.404120	0.524928	-0.31
859	1	-0.373896	-0.342195	-0.085227	-0.644600	-0.32

860 rows × 8 columns



▼ Sauvegarde du nouveau fichier formaté :

```
select_features(df_standardized).to_csv('CABANES_DECHENAIIS-PROJET.csv', index=False)
```

 ['Outcome', 'Direct_Bilirubin', 'Total_Bilirubin', 'Alkaline_Phosphotase', 'Age', 'Aspartate_Aminotransfere

	Outcome	Direct_Bilirubin	Total_Bilirubin	Alkaline_Phosphotase	Age	Aspartate_Aminotransfere
--	---------	------------------	-----------------	----------------------	-----	--------------------------